



HASHDIT

HashDit

Smart Contract Security

Venus BStockLiquidator Contract Security Audit Report

Venus

7 July 2026 – 10 July 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	7 July 2026 - 10 July 2026
Overall Risk [Optional]	Medium
Project Name	Venus
Code Base	https://github.com/VenusProtocol/venus-protocol/pull/683 https://app.notion.com/p/Bstock-Liquidaiton-Audit-Scope-392b4e5771bb80f885f6efc6ada9bdbf Fix: https://github.com/VenusProtocol/venus-protocol/pull/696/

Smart Contract List

No.	Contract Name	Verdict	Details
1	BStockLiquidator.sol	Medium	[M01]-Fixed [L01]-Ack [L02]-Fixed [L03]-Fixed [L07]-Fixed [I01]-Fixed
2	safe-fallback.ts	Low	[L04]-Fixed
3	atomic-liquidate.ts	Low	[L04]-Fixed [L06]-Fixed
4	bstock.ts	Low	[L04]-Fixed
5	019-deploy-bstock-liquidator.ts	Low	[L05]-Fixed

Findings

[M01] A borrower with VAI debt above the threshold cannot be liquidated by the backstop

Contract BStockLiquidator.sol

Severity Medium

The backstop routes its repay through the pool-wide Venus Liquidator, which refuses any non-VAI liquidation while the borrower's VAI debt is above minLiquidatableVAI, and this tool has no way to repay VAI, so a bStock borrower who also holds VAI debt cannot be cleared by the backstop, defeating its stated purpose of never letting the protocol carry bad debt on an RFQ-only asset.

Description

```
JavaScript
// BStockLiquidator.sol:333-368 (the ONLY repay path - no VAI branch)
IERC20Upgradeable debt = isBnb
    ? IERC20Upgradeable(address(wbnb))
    : IERC20Upgradeable(params.vDebt.underlying()); // @audit
vDebt=vaiController has no underlying() -> a VAI repay is impossible
here
...
debt.forceApprove(gate, params.repayAmount);
ILiquidator(gate).liquidateBorrow(
    address(params.vDebt),          // @audit always a normal market (e.g.
vUSDT), never vaiController
    params.borrower,
    params.repayAmount,
    params.vBStock
);

// Liquidator.sol:478-490 - the gate runs this on EVERY
liquidation, before any seize
function _checkForceVAILiquidate(address vToken_, address borrower_)
private view {
    uint256 _vaiDebt = vaiController.getVAIRepayAmount(borrower_);
    bool _isVAILiquidationPaused =
comptroller.actionPaused(address(vaiController),
IComptroller.Action.LIQUIDATE);
    bool _isForcedLiquidationEnabled =
comptroller.isForcedLiquidationEnabled(vToken_);
    if (
        _isForcedLiquidationEnabled ||
        _isVAILiquidationPaused ||
        !forceVAILiquidate ||
        _vaiDebt < minLiquidatableVAI ||
        vToken_ == address(vaiController)
    ) return;
```

```

    revert VAIDebtTooHigh(_vaiDebt, minLiquidatableVAI); // @audit
    reverts the whole backstop tx
}

```

Impact:

The contract's stated core guarantee ("the protocol never carries bad debt on an RFQ-only asset") fails for any bStock borrower who also carries VAI debt at or above the threshold — a state the borrower can create or maintain themselves. It is not a fund-loss bug: the position remains liquidatable through the standard two-step path (liquidate VAI first, then bStock). But that workaround is awkward and re-introduces the problem: if the borrower's only collateral is bStock, the VAI liquidation itself seizes bStock and hands the RFQ-only asset to whoever performs it. Severity is Medium rather than High because governance can bypass by enabling forced liquidation on the debt market (isForcedLiquidationEnabled(vDebt)), and off-chain ops can orchestrate a VAI-first clear; it is not Low because the guarantee fails under normal, borrower-inducible conditions and the path is entirely untested.

Recommendation

Add a VAI-aware path so the backstop is self-sufficient. Either

(a) add an operator entrypoint / preliminary hop that repays VAI through the gate with vToken = vaiController (satisfying the early-return at Liquidator.sol:487) and seizes+sells the bStock, driving VAI debt below minLiquidatableVAI before the main bStock repay; or, at minimum,

(b) detect the condition and surface a clear revert reason, and document that VAI must be cleared first, with monitoring for VAI-blocked bStock accounts.

Fixed in commit [6a21bca](#). Added VAI-debt recognition and repayment support, resolving vDebt via getVAIAddress() (VAIController has no underlying()) and rejecting FLASH mode for VAI (no vToken market to flash from):

Status

JavaScript

```

    IVAIController vaiCtrl = comptroller.vaiController();
    bool isVai = address(params.vDebt) == address(vaiCtrl);
    debt = isBnb
        ? IERC20Upgradeable(address(wbnb))
        : isVai
            ? IERC20Upgradeable(vaiCtrl.getVAIAddress())
            : IERC20Upgradeable(params.vDebt.underlying());

```

Note: this fix enables VAI liquidation but does not add a pre-flight check ensuring VAI debt is cleared before other debt when forceVAILiquidate is active — tracked separately as [L07].

[L01] Inventory Mode Has No Operator-Independent Floor on Swap Proceeds, Letting a Compromised Operator Drain the Contract

Contract BStockLiquidator.sol

Severity Low

In inventory mode the only lower bound on swap proceeds is the operator-supplied minOut, and the swap recipient lives inside operator-supplied calldata, so a compromised or rogue operator can repay real liquidations with the contract's inventory while routing the seized value to themselves — and the documented mitigation (minOut) is set by the very actor it is meant to constrain.

JavaScript

```
// BStockLiquidator.sol:196-198 (inventory entrypoint - only floor is minOut != 0)
_validateRouters(params.router, params.router2);
if (params.minOut == 0) revert ZeroMinOut(); // @audit any minOut >= 1 passes; no relation to repayAmount

// BStockLiquidator.sol:423-424 (sole proceeds check in inventory mode)
debtOut = debt.balanceOf(address(this)) - debtBefore;
if (debtOut < params.minOut) revert InsufficientOut(debtOut, params.minOut); // @audit floor is operator-chosen
```

Description

Worked example:

1. The contract is pre-funded with 1,000,000 USDT of protocol backstop capital.
2. A compromised operator key calls liquidate on a genuinely underwater borrower with repayAmount = 200,000 USDT, minOut = 1, and swapCalldata/swapCalldata2 that route the seized bStock's value (≈205,000 USDT after the treasury cut) to an operator-controlled counterparty (e.g. a thin/self-owned pool on an allowlisted AMM router with amountOutMin = 0, self-sandwiched), landing exactly 1 USDT back in the contract.
3. debtOut = 1 ≥ minOut = 1 passes. Inventory drops 1,000,000 → 800,001 USDT; ≈199,999 USDT is captured externally. Repeat until inventory is exhausted, before the owner can call setOperator(false).

Impact:

A single operator-key compromise drains the full debt-asset inventory in inventory mode (or skims the bonus every call in flash mode). Severity is Low under the project's trust model: operators are a trusted entity, so operator-only misuse is outside the threat model. It is retained (not dropped) because it is genuine defense-in-depth worth a cheap fix: operator keys are realistically hot automation keys (more exposed than the owner Safe), the contract's own NatSpec presents minOut as the blast-radius bound (which is circular against the operator), and inventory mode is asymmetric to flash

	mode, which already has an operator-independent floor. If the team ever treats operators as untrusted, this reverts to Medium.
Recommendation	Give inventory mode an operator-independent floor mirroring flash mode, e.g. require <code>debtOut >= repayAmount * minRecoveryBps / 10_000</code> with <code>minRecoveryBps</code> an owner-set (not operator-set) parameter. Optionally decode and constrain the swap recipient to <code>address(this)</code> rather than trusting caller <code>calldata</code> , and consider per-tx / cumulative-loss caps on the operator role.
Status	Acknowledged. Operators are a trusted role by design (the contract is operator-gated precisely because it custodies funds), and FLASH mode already enforces an operator-independent floor; the owner Safe can revoke a bad operator instantly via <code>setOperator</code> .

[L02] Partial RFQ fill strands the RFQ-only asset in the contract	
Contract	BStockLiquidator.sol
Severity	Low
Description	<code>_swap</code> approves an amount as a ceiling, not a floor, and only <code>minOut</code> (measured in the debt asset) is asserted — so if the MM's pre-signed quote sells less than the realized <code>seizedBStock</code>, the leftover <code>bStock</code> (or intermediate) is left in the contract, contradicting the NatSpec claim that the protocol "never ends up holding the RFQ-only asset." <pre> JavaScript // BStockLiquidator.sol:307-312 function _swap(IERC20Upgradeable token, address router, bytes memory data, uint256 amount) private { token.forceApprove(router, amount); // @audit ceiling, not floor - router may pull less (bool ok,) = router.call(data); if (!ok) revert SwapFailed(); token.forceApprove(router, 0); } // L423-424: only debtOut vs minOut is checked; no assertion that all seizedBStock was consumed </pre>
Recommendation	After the swap completes, check the seized <code>bStock</code>'s remaining balance in the contract. If nonzero, emit an event recording the leftover amount, so operations can recover it via the existing <code>sweep()</code> function. <pre> JavaScript // after the swap completes in _swap / _sellToDebt </pre>


```
uint256 leftover = token.balanceOf(address(this));
if (leftover > 0) {
    emit PartialSwapLeftover(address(token), leftover);
}
```

Fixed in commit [03bd5da](#). `_swap` now measures actual input consumed and emits `PartialSwapLeftover` on an underfill instead of silently stranding the residual:

Status

```
JavaScript
uint256 balBefore = token.balanceOf(address(this));
...
uint256 spent = balBefore - token.balanceOf(address(this));
if (spent < amount) emit PartialSwapLeftover(address(token), amount -
spent);
```

[L03] renounceOwnership is not overridden

Contract BStockLiquidator.sol

Severity Low

Description

The inherited `Ownable2StepUpgradeable.renounceOwnership()` remains callable and would set `owner = address(0)`, permanently disabling `sweep`, `sweepNative`, `setOperator`, and `setRouter` and stranding any inventory/native BNB in this fund-custodying contract.

Impact:

Owner self-harm only (not externally exploitable), but for a backstop that custodies protocol capital an accidental renounce is a permanent-loss footgun.

Recommendation

Override `renounceOwnership()` to revert (as the sibling `Liquidator.sol` already does).

Status

Fixed in commit [ce0b179](#). `renounceOwnership` overridden to a no-op so an accidental call cannot brick the fund-custodying contract:

```
JavaScript
function renounceOwnership() public override onlyOwner {}
```

[L04] liquidateCalculateSeizeTokens Called With the Wrong Function Overload

Contract scripts/bstock/safe-fallback.ts, scripts/bstock/atomic-liquidate.ts, tests/hardhat/Fork/helpers/bstock.ts

Severity Low

Venus's Comptroller exposes two overloads of liquidateCalculateSeizeTokens: a 3-argument version, which always reads the Core Pool's market parameters regardless of who the borrower is, and a 4-argument version that additionally takes a borrower parameter and reads the parameters of the pool that specific borrower actually belongs to. The only currently-active, non-legacy liquidation entry point, VToken.sol::liquidateBorrowFresh, calls the 4-argument, borrower-aware version. Three independent pieces of code introduced by this PR — the two operational scripts and the PR's own new fork-test helper — all call the 3-argument version instead, omitting the borrower entirely.

JavaScript

```
// VToken.sol - liquidateBorrowFresh (the version actually used
on-chain today)
(uint amountSeizeError, uint seizeTokens) =
comptroller.liquidateCalculateSeizeTokens(
    borrower,
    address(this),
    address(vTokenCollateral),
    actualRepayAmount
);
// safe-fallback.ts / atomic-liquidate.ts - the same defect in both
scripts
"function liquidateCalculateSeizeTokens(address,address,uint256) view
returns (uint256,uint256)",
const [seizeErr, seizeTokens] = await
comptroller.liquidateCalculateSeizeTokens(
    vDebt.address,
    vBStock.address,
    repay,    // missing the borrower argument
);
```

Description

A repo-wide cross-check confirms the only two legitimate uses of the 3-argument version elsewhere in the codebase — the legacy VTokenR1.sol (predating the multi-pool system, when the 4-argument version did not yet exist) and the fork test for BUSDLiquidator (whose underlying contract genuinely calls that same legacy path) — are each consistent with their own real on-chain behavior. This PR's three new call sites are the only ones applying that legacy-only function to a new contract (BStockLiquidator) that uses the current, non-legacy VToken.sol path.

This is not purely theoretical. A live query against BSC mainnet confirms Comptroller.lastPoolId() = 15 — 15 pools already exist beyond the Core Pool, several of which (LINK, UNI, AAVE, DOGE, BCH) are isActive = true with allowCorePoolFallback

= false. A real transaction executed on a mainnet fork confirms that a borrower holding sufficient other collateral can successfully call `enterPool()` to switch into any of these pools at will — the function's validation (`hasValidPoolBorrows`) only checks borrow-side compatibility, not collateral-side — with no governance approval required. The three `bStock` markets are currently listed only in the Core Pool (per VIP-633); once a borrower has performed such a switch, `getEffectiveLiquidationIncentive(borrower, vBStock)` reads a default value of zero (`bStock` unlisted in that pool, fallback disabled), while the 3-argument version continues to compute a nonzero value using Core Pool parameters — the two diverge.

Impact:

In `atomic-liquidate.ts`, the estimate only feeds the `swapCalldata` built for the Native quote; `BStockLiquidator.sol`'s `_swap` caps the router approval at the actually-measured on-chain seize amount rather than the script's estimate, so an inflated estimate simply causes the router's `transferFrom` to exceed its allowance and revert with `SwapFailed()` — a failed transaction, not a fund loss. In `safe-fallback.ts`, the estimate is hard-coded into the batch's `redeem()` step; if the batch is executed atomically as intended via Transaction Builder, an inflated estimate causes `redeem()` to revert (attempting to redeem more than the batch actually received earlier in the same execution), rolling back the earlier repay step as well, so the batch fails as a whole rather than producing a net loss — provided the signer does not deviate from standard usage by splitting the batch into separate transactions. Separately, because the PR's own fork-test helper shares this same defect, the test suite validates its estimate against an equally-wrong reference and never exercises the cross-pool scenario, which is the direct reason this defect was not caught during testing.

Update all three call sites to use the 4-argument, borrower-aware overload:

JavaScript

```
comptroller["liquidateCalculateSeizeTokens(address,address,address,uint256)"](  
    borrower, vDebt.address, vBStock.address, repay  
)
```

Recommendation

Add a `require(seizeTokens.gt(0))`-style sanity assertion at the estimation stage so anomalies surface early rather than at execution time. Also update `tests/hardhat/Fork/helpers/bstock.ts` and add a fork test case covering a borrower in a non-Core-Pool state, so the test suite and production scripts no longer share — and mutually mask — the same defect.

Fixed in commit [355b16b](#). All three call sites switched to the borrower-aware 4-arg `liquidateCalculateSeizeTokens` overload, plus a zero-seize sanity guard:

JavaScript

```
const [seizeErr, seizeTokens] = await  
comptroller.liquidateCalculateSeizeTokens(  
    borrower, vDebt.address, vBStock.address, repay,  
);  
if (seizeTokens.eq(0)) throw new Error(`liquidateCalculateSeizeTokens  
returned 0 seize for ${borrower}`);
```

Status

[L05] Deploy Script's Post-Deploy Checklist Omits the authorizedFlashLoan Governance Step, Leaving flashLiquidate Unusable

Contract deploy/019-deploy-bstock-liquidator.ts

Severity Low

Venus's FlashLoanFacet requires that any contract initiating a flash loan first be added to the authorizedFlashLoan whitelist by the Comptroller admin via a governance action, or the call reverts. The deploy script's POST-DEPLOY comment lists only three actions – setRouter (twice) and setOperator – and does not mention this additional, separately-governed step. The requirement is documented in the same PR's IBStockLiquidator.sol interface comments, but was not carried over into the deploy script's own checklist.

JavaScript

```
// deploy/019-deploy-bstock-liquidator.ts
// POST-DEPLOY (executed by the owner Safe, since it owns the contract
// from t0):
// 1. setRouter(nativeRfqRouter, true)
// 2. setRouter(ammRouter, true)
// 3. setOperator(operatorKey, true)
// (missing: Comptroller admin must separately whitelist this contract
// via authorizedFlashLoan)

// FlashLoanFacet.sol - executeFlashLoan
if (!authorizedFlashLoan[msg.sender]) {
    revert SenderNotAuthorizedForFlashLoan(msg.sender);
}
```

Description

A live on-chain query against the already-deployed contract on BSC mainnet confirms:
cast call 0xfD36...8384 "authorizedFlashLoan(address)(bool)" 0xF03C90...4A3c1
→ false

This is the deployed contract's actual current state, not a theoretical projection.

Impact

FLASH funding mode is entirely non-functional in the current deployment state; flashLiquidate reverts early in the flash loan flow. There is no fund-loss risk – INVENTORY mode and all admin functions are unaffected – and the condition is fully reversible: the Comptroller admin can grant the authorization at any time via a single governance action (setWhiteListFlashLoanAccount), with no contract redeployment required. The only cost is that, absent this fix, operations staff may first discover the missing authorization during an actual emergency liquidation that requires FLASH mode, wasting time under pressure.

Recommendation	Add "coordinate with the Comptroller admin to authorize this contract via setWhiteListFlashLoanAccount" to the deploy script's POST-DEPLOY checklist comment as a pre-launch requirement rather than something discovered after the fact. Fixed in commit a2d2043 . Deploy script's POST-DEPLOY checklist now documents the governance-gated whitelist step:
Status	JavaScript <pre>// 4. Comptroller.setWhiteListFlashLoanAccount(bStockLiquidator, true) - via the Comptroller admin</pre>

[L06] Two-Hop Path Doesn't Re-Verify Native Quote TTL Before Submission	
Contract	scripts/bstock/atomic-liquidate.ts (off-chain settlement script paired with BStockLiquidator.sol)
Severity	Low
Description	After obtaining a firm quote from the Native market maker, the script checks the quote's remaining TTL exactly once. For a two-hop path, it then performs several additional steps that consume real wall-clock time before submission: an AMM aggregator API round-trip for the second hop, an on-chain isRouter read-only check for both routers, and finally submitting and waiting for the settlement transaction to be mined. The TTL is never re-checked after the initial check, right up to the point of submission. JavaScript <pre>// atomic-liquidate.ts const q = await getFirmQuote({ tokenIn: bStock.address, tokenOut: nativeOut, amount: seizedHumanQuote, slippage }); const ttl = quoteDeadline(q) - Math.floor(Date.now() / 1000); if (ttl <= 0) throw new Error("quote already expired - refetch"); deadline = BigNumber.from(quoteDeadline(q)); ... // Hop 2: AMM aggregator API round-trip (unbounded latency) const amm = await getAmmSwap({ tokenIn: nativeOut, tokenOut: debt.address, amountIn: midOut.toString(), ... }); ... // isRouter on-chain read check for (const r of routers) { if (!(await liquidator.isRouter(r))) throw new Error(...); } ... // Submission - ttl is never re-checked here const tx = await liquidator[fn](params);</pre>

	The contract does provide a backstop: liquidate/flashLiquidate compares the on-chain block timestamp against the submitted deadline and reverts if it has passed, so an expired quote can never be force-settled. Impact: No fund loss is possible — the on-chain DeadlineExpired check catches any genuinely expired quote before settlement. The real-world impact is limited to a wasted liquidation attempt: if the AMM aggregator API or the RPC node responds slowly, the quote may expire between the initial TTL check and final submission, causing the transaction to revert, burning gas and requiring the operator to refetch and retry. The trigger condition is external-dependency slowness rather than BSC network congestion (BSC's current block time is fast enough that on-chain queuing contributes negligible delay), making this a low-probability edge case rather than a structural issue.
Recommendation	Recompute the remaining TTL immediately before the final call to liquidate/flashLiquidate, and abort and refetch a fresh quote if the remaining TTL has dropped below a safety margin, rather than relying entirely on the on-chain deadline check as the sole backstop.
Status	Fixed in commit 4d176db. Re-verifies the Native quote's remaining TTL immediately before submission, aborting below a configurable margin: <pre>JavaScript const remainingTtl = deadline.toNumber() - Math.floor(Date.now() / 1000); if (remainingTtl < settleTtlMarginSec) throw new Error(`Native quote TTL \${remainingTtl}s is below the \${settleTtlMarginSec}s safety margin - refetch`);</pre>

[I01] sweepNative has no nonReentrant guard	
Contract	BStockLiquidator.sol
Severity	Informational
Description	sweepNative sends native BNB with a low-level call and, unlike the liquidation entrypoints, carries no nonReentrant guard; this was investigated and found non-exploitable, so the recommendation is consistency-only. <pre>JavaScript // BStockLiquidator.sol:175-180 function sweepNative(address to, uint256 amount) external override onlyOwner { // @audit-notice sweepNative has no reentrancy protection ensureNonzeroAddress(to); (bool ok,) = to.call{ value: amount }(""); // @audit external call, no reentrancy lock held if (!ok) revert NativeTransferFailed();</pre>

```
emit SweptNative(to, amount);
}
```

Impact:

None demonstrable. The function is onlyOwner, snapshots no state, and reads no balance after the call, so a re-entrant recipient cannot corrupt any accounting; the swept BNB is already sent; and any re-entry into liquidate/flashLiquidate is an atomic, minOut-bounded action the caller (an operator) is already authorized to perform. The contract holds no stored ledger for a reentrant call to desync.

Recommendation Add nonReentrant for consistency with liquidate/flashLiquidate (defense-in-depth); no functional change required.

Status Fixed in commit [4617af5](#). Added nonReentrant for consistency with the liquidation entrypoints:

```
JavaScript
function sweepNative(address to, uint256 amount) external override
onlyOwner nonReentrant {
```

New Finding

[L07] Scripts Don't Detect the VAI-Must-Clear-First Precondition, Risking a Failed Liquidation of Other Debt

Contract BStockLiquidator.sol

Severity Low

Description Liquidator.sol's pre-existing _checkForceVAILiquidate logic blocks liquidating any non-VAI debt market when forceVAILiquidate is enabled and the borrower's VAI debt is at or above minLiquidatableVAI, unless VAI is cleared below that threshold first. The M-1 fix gave BStockLiquidator the ability to liquidate VAI debt, but added no corresponding pre-flight check in the contract or the off-chain scripts — _liquidate's isVai flag only decides how the debt asset is resolved for the current call; it does not check upfront whether the borrower's overall VAI exposure will cause this (non-VAI) liquidation to be blocked.

```
JavaScript
// BStockLiquidator.sol - _liquidate()
IVAIController vaiCtrl = comptroller.vaiController();
bool isVai = address(params.vDebt) == address(vaiCtrl);
// isVai only affects how the debt asset is resolved - it does not
pre-check whether the
```

```

// borrower's VAI exposure exceeds the threshold
...
ILiquidator(gate).liquidateBorrow(address(params.vDebt),
params.borrower, params.repayAmount, params.vBStock);
// The actual block happens inside Liquidator.sol's private
_checkForceVAILiquidate, which
// BStockLiquidator has no visibility into ahead of time

```

Neither `atomic-liquidate.ts` nor `safe-fallback.ts` queries `forceVAILiquidate()` / `getVAIRepayAmount(borrower)` / `minLiquidatableVAI()` before constructing a non-VAI liquidation transaction.

Impact:

No fund-loss risk, and the position never becomes permanently unliquidatable — liquidating the VAI debt itself is never blocked by this check (`vToken_ == address(vaiController)` bypasses it unconditionally), and `BStockLiquidator`, following the M-1 fix, is now capable of independently completing the "clear VAI, then clear the rest" two-step sequence. The condition is currently dormant: an on-chain check against the BSC mainnet Liquidator gate (`0x0870793286aaDA55D39CE7f82fb2766e8004cF43`) confirms `forceVAILiquidate()` is currently false, so this check is not presently triggered. If governance ever enables it, attempting a non-VAI liquidation against a borrower whose VAI debt exceeds the threshold will revert on-chain (`VAIDebtTooHigh`), requiring the operator to manually retry by liquidating VAI first — a wasted transaction and an operational delay.

Add a pre-flight check in `atomic-liquidate.ts`/`safe-fallback.ts` before submitting a non-VAI liquidation:

JavaScript

```

if (vDebt.address !== vaiControllerAddress) {
  const forceOn: boolean = await gate.forceVAILiquidate();
  if (forceOn) {
    const vaiDebt: BigNumber = await
vaiController.getVAIRepayAmount(borrower);
    const minLiquidatableVAI: BigNumber = await
gate.minLiquidatableVAI();
    if (vaiDebt.gte(minLiquidatableVAI)) {
      throw new Error(`Borrower has ${vaiDebt} VAI debt >= threshold
${minLiquidatableVAI} - liquidate VAI first`);
    }
  }
}

```

Recommendation

Further, the scripts could automatically construct and submit a VAI liquidation first when this condition is detected, rather than relying on the operator to remember the correct order; `safe-fallback.ts` could go further still and bundle both steps into the same `Safe MultiSend` batch for atomic execution.

Status

Fixed in commit [1507c56](#). Added `lib/vai-gate.ts::assertVaiGateClear()`, a pre-flight that mirrors all five terms of `Liquidator._checkForceVAILiquidate` (not just the debt threshold), called from both `atomic-liquidate.ts` and `safe-fallback.ts` before

submission/batch-build, naming the VAI-first remedy instead of burning a tx on an on-chain revert:

JavaScript

```
    if (a.vDebt.toLowerCase() === a.vaiController.toLowerCase()) return;
    // liquidating VAI itself is never blocked
    if (!(await gate.forceVAILiquidate())) return;
    // global switch off (mainnet default)
    ...
    throw new Error(`...Liquidate the VAI debt FIRST
(VDEBT=${a.vaiController})...`);
```